
Prerequisites

Note: All of the following are completed in the GitHub Workshop

- Have the following: GitHub account, VS Code and Hugo installed
    - See setup guide here
    - You can run `hugo version` in a VS Code terminal to ensure it's installed

- Have Git installed and authenticate VS Code with GitHub (if you haven't already)
    - To install git, see here
    - To authenticate VS Code with GitHub, see here

- Fork (on GitHub) and clone the GWC website repo (in VS Code)
    - For more on how to fork, see here

    - For cloning the GWC website repo, clone recursively
        - *Generally, you can clone as shown in the GitHub workshop, but our website uses another repository for the Hugo theme (submodule).*

    - **[UPDATE]** After cloning, run this command in your VS Code terminal:
      `git checkout -b hugo-workshop 4e696588744c83d7e4593c585dd2e74395180089`
        - This gives you the version of the repo before Girls Who Code used it to build their current website

  Note: If you already cloned before the repo was updated (right after the GitHub Workshop), run this command in your VS Code terminal.
  OR You can delete the folder you saved locally and your fork in GitHub, then fork again.

Hugo

What is it?

Hugo's website says that it is a "popular open-source static site generator" (src), but what is a static site generator?

How does it work?

A static site generator (SSG) is a tool that generates a static site (only html, css, js) for you in a specified format. Compared to a dynamic site (i.e. wordpress) it allows you to have a more simplistic/minimal approach to building a website. Instead of having to deal with the learning overhead of understanding a database, backend server side language (i.e. php, node, etc..), etc (which normally a lot of dynamic sites include those), static sites are normally just text files. A SSG can allow you to not have to code everything yourself as well.

You can use programming logic (if statements, for loops, etc..) and shortcodes (built-in functions) to also help you with writing for a SSG so you can reuse pieces of html on different pages and not have to type as much. They also normally allow you to write content in different

markup languages (i.e. markdown) instead of having to code all the content as HTML for every page. SSGs are a nice blend between a full blown programming language and plain text.

Overview of Roxo Theme

Folder Structure

If you look at the original Roxo theme's repo, you will see that our repo doesn't look the same. This is because their repo is formatted to be used as a theme, and we have that theme as a folder in our repo. We have copied the contents of the exampleSite folder from the theme to the root of our repo. This allowed us to set up their example site as our own that will use the roxo theme but can override files and styles we want to change.

If you look in the themes folder, you will see the contents and structure of that original repo. We only copy files out from the themes folder to change and override things inside that themes folder. Until there's a need to do that for any particular file, you should leave them in the themes folder.**Never modify the files in the themes folder directly. Always copy them outside of the folder to the same path in our repo and modify them there.**
Note: We need to copy them to the same path in our repo for them to override that same file in the themes folder. (For example, layouts/index.html in ours overrides the themes/roxo folder path of layouts/index.html)

Config File

The config.toml for our repo (outside of the themes folder) instructs hugo on how we want things configured, it specifies the theme to use and where it's located.

Git Files

Your git client is almost like your browser, but instead of accessing websites you're accessing git repos. All the files that start with .git change your git client's settings for how it is supposed to interact with this repo.

- .gitignore
    - This is where we can specify what we don't want to include in the repo
    - We use this for files we don't want if they are dynamically generated files (i.e. they are generated every time the hugo server is run - so we can't just delete them)

- .gitattributes
    - can use to specify how you want to read/interpret certain attributes of your files inside of the git repo
    - currently being used to prevent unnecessary modifications when Windows saves text files (Windows doesn't have the same line endings compared to the more widely accepted line ending format - won't go into this here, but just for an example)

- .gitmodules
    - Can use to specify what other repos you want to include

    - Currently being used to specify the roxo theme repo (which is a submodule then)

Forestry

- Not currently being used but was offered as a default part of the example site
- Won't really go into this, but you can learn more if you're interested here
- Company to help abstract static site generators (i.e. put a GUI on top)
    - Content management system for people who don't like dealing with code

Archetypes

- Archetype files are used for templating content markdown files
    - For example, if you had certain details you always wanted to include for blog posts (author name, date, etc.), you could template that out in an archetype markdown file
- Currently includes: default.md

Assets (Styling)

- **CSS**
    - **Bootstrap**
        - Explanation (src):

> popular framework for building responsive, mobile-first sites

- **JS**
    - vendor.js - jQuery (src)

> jQuery is a fast, small, and feature-rich JavaScript library.

    - formhandler.js - custom AJAX (part of jQuery) request for form submissions (src)

> AJAX allows web pages to be updated asynchronously by exchanging data with a web server behind the scenes. This means that it is possible to update parts of a web page, without reloading the whole page.

    - script.js - custom javascript file to handle scrolling (extended from jquery)

- **SCSS** (Sassy CSS) (src)

> Sass is a stylesheet language that's compiled to CSS. It allows you to use variables, nested rules, mixins, functions, and more, all with a fully CSS-compatible syntax.

Content

- Content of main pages, written in markdown
    - For more on syntax, see a markdown cheat sheet
- Each page category gets its own folder
    - Currently includes: About, Blog, Contact, Portfolio

Data

- Contains main data used for websites (like a database), written in YAML
- The html files in the themes/roxo/layouts folder can read the data from these
    - For example, data/teams.yml contains information about each team member
    - Then, themes/roxo/layouts/partials/team.html pulls from that information using a range of all the members (like a loop) that displays each of the members and their corresponding information.

Layouts

- Used to specify the structure of different webpages, written in HTML
    - Can be main webpages and include partials
        - For example, themes/roxo/layouts/about/list.html uses partials: clients.html, team.html, and call-to-action.html
- Currently includes: 404.html and index.html
    - 404 is used when a page is not found
    - index is the home page

Resources

- Grayed out because it is mentioned in the .gitignore file
    - Note: You will only see this folder once you have run the hugo server command
- We are currently ignoring this folder because it's automatically generated assets that we don't need to commit to the repo

Static

- **Images**
    - Files organized by categories of folders
- Note that inside the themes folder, the static folder has css and font-awesome folders
    - If you want to know more about font-awesome, see here

Config

- Instructs hugo on how we want things configured, written in TOML
- NOTE: You don't want to make changes in ours unless it's absolutely necessary
    - You don't want to make changes to the themes/roxo/config.toml at all
- This is where we can specify routes
    - For example, where a form submissions go or the menu tabs and pages
- We can also specify other params like persistent pieces
    - For example, social media links in the footer

Want to know more?

You can check out Mike Dane's video on directory structure with Hugo.

Ready to play with the site?

Check out the slides from our Let's Play with Hugo! Workshop here.
If you already have everything set up (see prerequisites), you can start on slide 8!